17. Ensemble Models

Random Forest • Gradient Boosting • XGBoost • LightGBM • CatBoost

Veri Bilimci Müfredatı

2026

Modülün amacı

Ensemble learning, tek bir model yerine birden fazla modelin tahminlerini birleştirerek daha güçlü, daha stabil ve çoğu zaman daha iyi genelleyen sistemler üretir. Bu modülde amaç yalnızca `RandomForestClassifier()` veya `XGBClassifier()` çağırmayı öğrenmek değil; **neden ensemble işe yarar, hangi ensemble ailesi hangi hatayı azaltır, tuning hangi sırayla yapılır ve sonuç nasıl yorumlanır** sorularını cevaplayabilmektir.

Bu modül sonunda:

- Bagging, boosting, voting ve stacking farklarını açıklayabileceksin.
- Random Forest'ın bootstrap ve random feature sampling mantığını anlayacaksın.
- Gradient Boosting'in residual/gradient düzeltme yaklaşımını kavrayacaksın.
- XGBoost, LightGBM ve CatBoost arasındaki pratik farkları bileceksin.
- `learning_rate`, `n_estimators`, `max_depth`, `num_leaves`, `subsample`, `colsample_bytree` ve regularization parametrelerini sistematik tune edebileceksin.
- Feature importance, permutation importance ve SHAP sonuçlarını doğru sınırlamalarla yorumlayabileceksin.

1. Ensemble learning nedir?

Bir modelin hatası kabaca iki ana parçadan düşünülebilir: **bias** ve **variance**. Tek bir model bazı bölgelerde sistematik olarak yanlış olabilir veya training sample değiştiğinde tahminleri aşırı oynayabilir. Ensemble yaklaşımı, modellerin hatalarının birebir aynı olmamasından yararlanır.

Yaklaşım	Eğitim şekli	Birleştirme	Ana hedef
Bagging	Paralel / bağımsız	Ortalama veya voting	Variance azaltmak
Boosting	Ardışık	Ağırlıklı toplam	Bias ve variance azaltmak
Voting	Farklı modeller	Oy / probability ortalaması	Model çeşitliliği
Stacking	Farklı modeller	Meta-model	Tahminleri öğrenerek birleştirmek

Ana sezgi: Ensemble'ın gücü sadece "çok model" kullanmak değildir. Güç, modellerin **farklı hata desenlerine** sahip olmasından gelir.

2. Bias-variance açısından ensemble

Derin Decision Tree çoğu zaman düşük bias ama yüksek variance taşır. Training set biraz değiştiğinde ağacın split yapısı ciddi değişebilir. Bu, bagging için ideal bir durumdur.

Model	Bias	Variance
Sığ Decision Tree	Yüksek	Düşük
Derin Decision Tree	Düşük	Yüksek
Random Forest	Düşük-orta	Tek ağaca göre daha düşük
Gradient Boosting	Düşük olabilir	Regularization yoksa yükselebilir

Random Forest ağırlıklı olarak **variance azaltır**. Boosting ise zayıf modelleri ardışık düzelterek bias'ı güçlü biçimde düşürebilir.

3. Voting ve averaging

En basit ensemble, aynı problem için birkaç modelin tahminlerini ortalamaktır.

```
# Regression
pred = (pred_model1 + pred_model2 + pred_model3) / 3

# Weighted averaging
pred = 0.5 * pred1 + 0.3 * pred2 + 0.2 * pred3
```

Classification'da iki temel yaklaşım vardır:

- **Hard voting:** Her model sınıf oyu verir.
- **Soft voting:** Modellerin sınıf olasılıkları ortalanır.

Soft voting, probability kalitesi iyi modellerde daha esnektir.

4. Bagging — Bootstrap Aggregating

Bagging şu akışla çalışır:

1. Training setten replacement ile bootstrap sample çek.
2. Her sample üzerinde ayrı base model eğit.
3. Tahminleri birleştir.
4. Regression'da ortalama, classification'da voting/probability average kullan.

Neden işe yarar? Modeller tamamen aynı hatayı yapmıyorsa ortalama alma rastgele dalgalanmayı azaltır.

4.1 Bootstrap sampling

n gözlemden replacement ile yine n gözlem çektiğinde bazı kayıtlar birden fazla kez gelir, bazıları hiç gelmez. Büyük örneklemde bootstrap sample yaklaşık olarak orijinal gözlemlerin %63.2 kadar farklı gözlemini içerir.

Seçilmeyen kayıtlar **Out-of-Bag (OOB)** örneklerdir.

5. Random Forest

Random Forest, bagging'e iki önemli rastgelelik ekler:

- Satırlar için bootstrap sampling
- Her split sırasında feature'ların rastgele alt kümesini deneme

İkinci mekanizma ağaçlar arasındaki korelasyonu düşürür. Ağaçlar daha farklı hale gelir, averaging daha etkili olur.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=500,
    max_depth=None,
    min_samples_leaf=5,
    max_features="sqrt",
    class_weight="balanced",
    random_state=42,
    n_jobs=-1
)
```

6. Random Forest hyperparameter'ları

Parametre	Etki
`n_estimators`	Ağaç sayısı; daha fazla genellikle daha stabil ama daha pahalı
`max_depth`	Her ağacın karmaşıklığını sınırlar
`min_samples_leaf`	Leaf başına minimum örnek; smoothing sağlar
`max_features`	Ağaçlar arası korelasyon ile split gücü arasında denge
`bootstrap`	Satır sampling yaklaşımı
`class_weight`	Imbalanced classification'da sınıf hatalarının önemini değiştirir

Tuning sırası: Önce validation design ve metric; sonra tree complexity; sonra `max_features`; en sonda `n_estimators` yeterli stabiliteye çıkarılır.

7. OOB evaluation

Random Forest'ta bir ağacın bootstrap sample'ına girmeyen kayıtlar o ağaç için küçük bir internal validation seti gibi kullanılabilir.

```
rf = RandomForestClassifier(
    n_estimators=500,
    bootstrap=True,
    oob_score=True,
    random_state=42
)
```

OOB skoru faydalı bir sinyaldir ancak **gerçek temporal/group split'in yerine geçmez**.

8. Extra Trees

Extra Trees, Random Forest'tan daha fazla rastgelelik ekler. Split threshold'ları daha rastgele seçilebilir. Bu yaklaşım:

- Training'i hızlandırabilir,
- Tree correlation'ını azaltabilir,
- Bir miktar ekstra bias karşılığında variance'ı azaltabilir.

9. Boosting fikri

Bagging'de modeller bağımsızdır. Boosting'de ise modeller **ardışık** çalışır.

Zihinsel model:

> Önce kaba bir model kur. Sonra bu modelin yaptığı hatayı öğrenen yeni bir model ekle. Kalan hatayı tekrar düzelt. Bu işlemi kontrollü biçimde sürdür.

Bu yüzden boosting, küçük ve zayıf ağaçları art arda ekleyerek çok güçlü bir model oluşturabilir.

10. Gradient Boosting

Gradient Boosting, loss fonksiyonunu function space içinde gradient descent benzeri adımlarla optimize eder. Her yeni tree, mevcut ensemble'ın negative gradient'ini veya özel durumlarda residual'ını yaklaşıklar.

Regression squared error durumunda sezgi çok nettir:

1. İlk tahmini üret.
2. Residual $r = y - \hat{y}$ hesapla.
3. Residual'ı tahmin eden yeni tree eğit.
4. Yeni tree'nin katkısını `learning_rate` ile küçült.
5. Ensemble'a ekle.

```
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)
```

11. learning_rate ve n_estimators ilişkisi

Bu iki parametre birlikte düşünülmelidir.

Kombinasyon	Tipik davranış
Büyük LR + az tree	Hızlı öğrenir; overfit/overshoot riski
Küçük LR + çok tree	Daha yavaş; çoğu zaman daha kontrollü
Çok küçük LR	Eğitim maliyeti gereksiz büyüyebilir

`learning_rate`, her yeni learner'ın ensemble'a katkısını küçülten bir **shrinkage** mekanizmasıdır.

12. Stochastic Gradient Boosting

Boosting round'larında satırların veya sütunların tamamı yerine rastgele alt kümeleri kullanılabilir.

```
subsample = 0.8
colsample_bytree = 0.8
```

Bu, modele stochasticity ekler ve overfitting'i azaltabilir.

13. XGBoost

XGBoost, Gradient Boosted Decision Trees yaklaşımını güçlü regularization ve mühendislik optimizasyonlarıyla genişletir.

Öne çıkan özellikler:

- First-order gradient yanında second-order Hessian bilgisi
- L1/L2 regularization
- Row ve column subsampling
- Sparse/missing değer handling
- Parallel split search
- Early stopping

```
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=2000,
    learning_rate=0.03,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.0,
    reg_lambda=1.0,
    eval_metric="logloss",
    random_state=42
)
```

13.1 XGBoost'ta önemli parametreler

Grup	Parametreler
Complexity	`max_depth`, `min_child_weight`
Learning	`learning_rate`, `n_estimators`
Sampling	`subsample`, `colsample_bytree`
Regularization	`reg_alpha`, `reg_lambda`, `gamma`
Imbalance	`scale_pos_weight`

14. Early stopping

Boosting'de ideal tree sayısını önceden bilmek zordur. Validation metric iyileşmeyi bıraktığında training'i durdurmak overfit'i sınırlar.

Önemli kural:

> Early stopping için kullanılan validation set **final test set olmamalıdır**.

15. LightGBM

LightGBM özellikle büyük tabular datasetlerde hız ve bellek verimliliğiyle öne çıkar.

Ana fikirler:

- Histogram tabanlı split search
- Leaf-wise tree growth
- Native categorical seçenekleri
- Büyük veri için yüksek hız

Leaf-wise büyüme çok güçlüdür ancak küçük datasetlerde agresif `num_leaves` / düşük `min_data_in_leaf` overfitting'e yol açabilir.

16. CatBoost

CatBoost özellikle kategorik feature'lar için tasarlanmıştır.

Önemli fikirler:

- Native categorical feature handling
- Ordered target statistics
- Ordered boosting
- Güçlü default parametreler
- Target leakage riskini azaltan categorical encoding yaklaşımı

```
from catboost import CatBoostClassifier

model = CatBoostClassifier(
    iterations=1200,
    learning_rate=0.05,
    depth=7,
    loss_function="Logloss",
    verbose=False,
    random_seed=42
)
```

17. XGBoost vs LightGBM vs CatBoost

Model	Güçlü olduğu durum	Dikkat
XGBoost	Güçlü genel tabular baseline, geniş ecosystem	Tuning alanı geniş
LightGBM	Büyük dataset, hızlı iteration	Leaf-wise overfit riski
CatBoost	Kategorik feature yoğun veri	Bazı senaryolarda daha yavaş training

Tek bir "en iyi" boosting kütüphanesi yoktur. Aynı split ve metric altında benchmark yapılmalıdır.

18. Regression ve classification ensemble'ları

Task	Örnek modeller	Output
Regression	RandomForestRegressor, XGBRegressor, LGBMRegressor	Sayısal tahmin
Classification	RandomForestClassifier, XGBClassifier, CatBoostClassifier	Probability / class

19. Imbalanced classification

Rare positive problemlerinde ensemble algoritması seçmek tek başına yeterli değildir.

Kontrol edilmesi gerekenler:

- ROC-AUC yanında **PR-AUC**
- `class_weight` / `scale_pos_weight`
- Sampling yalnız training fold içinde
- Threshold tuning
- False Positive / False Negative business maliyetleri

Örneğin XGBoost'ta başlangıç için:

$$\text{scale_pos_weight} \approx \frac{N_{negative}}{N_{positive}}$$

kullanılabilir, ancak validation ile doğrulanmalıdır.

20. Feature importance

Tree ensemble'larda impurity/gain tabanlı importance kolay elde edilir ancak dikkatli yorumlanmalıdır.

Yöntem	Güçlü yön	Risk
Impurity/Gain	Çok hızlı	High-cardinality bias
Permutation Importance	Model-agnostic	Correlated feature'larda önem bölünebilir
SHAP	Local + global açıklama	Hesaplama ve yorum maliyeti

Feature importance nedensel önem değildir.

21. Permutation Importance

Bir feature'ın değerleri validation setinde karıştırıldığında metric ne kadar bozuluyor?

```
from sklearn.inspection import permutation_importance

r = permutation_importance(
    model,
    X_valid,
    y_valid,
    scoring="roc_auc",
    n_repeats=10,
    random_state=42
)
```

Bu yöntem feature'ın model performansına katkısını daha doğrudan ölçer.

22. SHAP

SHAP bir prediction'ı baseline tahminden feature katkılarına ayırır.

Kullanım katmanları:

- **Global:** mean absolute SHAP ile genel feature importance
- **Local:** tek müşteri/işlem için contribution analizi
- **Dependence:** feature değeri değiştikçe katkı nasıl değişiyor?

SHAP model davranışını açıklar; gerçek dünyanın causal mekanizmasını değil.

23. Hyperparameter tuning stratejisi

Doğru tuning sırası:

1. Split tasarımını sabitle.
2. Metric'i sabitle.
3. Baseline oluştur.
4. Complexity parametrelerini ayarla.
5. `learning_rate` + `n_estimators` dengesini ayarla.
6. Sampling parametrelerini ekle.
7. Regularization tune et.
8. Early stopping kullan.
9. Test seti yalnız finalde aç.

23.1 Parametre grupları

Grup	Örnekler
Complexity	`max_depth`, `num_leaves`, `min_child_weight`
Learning	`learning_rate`, `n_estimators`
Sampling	`subsample`, `colsample_bytree`
Regularization	`reg_alpha`, `reg_lambda`
Imbalance	`class_weight`, `scale_pos_weight`

24. Random search, grid search ve Optuna

Grid search her kombinasyonu dener. Hyperparameter uzayı büyüdükçe maliyet hızla artar.

Random search çoğu zaman daha verimlidir çünkü önemli boyutlarda daha fazla farklı değer görür.

```python
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    estimator=model,
    param_distributions=params,
    n_iter=50,
    scoring="roc_auc",
    cv=5,
    random_state=42,
    n_jobs=-1
)
```

Optuna gibi yöntemler önceki trial sonuçlarını kullanarak gelecek denemeleri daha akıllı seçebilir.

25. Leakage ve ensemble modeller

Güçlü modeller leakage'i daha da iyi öğrenir. Bu nedenle validation disiplini kritik önemdedir.

Hata	Sonuç
Split öncesi target encoding	Validation target bilgisi sızar
Future aggregates	Gelecek bilgi feature olur
Test set ile early stopping	Test unbiased olmaktan çıkar
Split öncesi SMOTE	Synthetic leakage oluşur
Test üzerinde tekrar tekrar model kararı	Test overfitting

26. Cross-validation ve zaman

Veri yapısı	Önerilen validation
IID	KFold / StratifiedKFold
Aynı entity tekrarları	GroupKFold
Zaman bağımlı	TimeSeriesSplit / temporal folds
Fraud/churn event stream	Temporal veya purged design

En iyi boosting modeli yanlış split ile gerçekte başarısız olabilir.

27. Model comparison matrix

Model	Hız	Nonlinearity	Scaling	Categorical	Typical kullanım
Random Forest	Orta	Güçlü	Gerekmez	Encode gerekir	Robust baseline
XGBoost	Orta	Çok güçlü	Gerekmez	Genellikle encode	Genel tabular SOTA adayı
LightGBM	Hızlı	Çok güçlü	Gerekmez	Native seçenek	Büyük veri
CatBoost	Orta	Çok güçlü	Gerekmez	Çok güçlü native	Kategorik veri

28. Gerçek vaka — Churn ensemble benchmark

Senaryo: Bir abonelik şirketi önümüzdeki 30 gün içinde churn edecek müşterileri belirlemek istiyor.

Aşama	Yaklaşım
Baseline	Logistic Regression
Tree baseline	Decision Tree
Bagging	Random Forest
Boosting	XGBoost / LightGBM / CatBoost
Split	Temporal
Metric	PR-AUC + recall@capacity + calibration
Threshold	Retention ekibinin günlük kapasitesi
Explainability	Permutation + SHAP + error analysis

Bu vaka portföy için güçlüdür çünkü basit modelden modern boosting'e performans ve complexity artışını görünür kılar.

29. Gerçek vaka — Kredi default tahmini

Kredi riskinde sadece AUC yeterli değildir.

Kontroller:

- Default probability calibration
- Temporal stability
- Segment bazlı hata
- Fairness
- Explainability
- Regulatory constraints
- Threshold ve expected loss

False negative burada riskli müşteriye kredi verme anlamına gelebilir. Hata maliyeti asymmetric'tir.

30. Production açısından ensemble

En iyi offline score her zaman en iyi production model değildir.

Değerlendir:

- Model boyutu
- Inference latency
- Batch vs online scoring
- Memory kullanımı
- Dependency karmaşıklığı
- CPU/GPU ihtiyacı
- Explainability latency
- Retraining maliyeti

Random Forest 1,500 ağaçla 0.001 AUC daha iyi olabilir; ama latency 10 kat artıyorsa production için daha küçük model daha doğru olabilir.

31. Sık yapılan hatalar

- Sadece `n_estimators` yükseltmeyi tuning sanmak.
- Validation design kötü iken model tuning yapmak.
- ROC-AUC'yi tek metric olarak kullanmak.
- Feature importance'ı causal yorumlamak.
- Test seti early stopping veya threshold tuning'de kullanmak.
- Kategorik encoding leakage'ını göz ardı etmek.
- Random Forest scaling istemiyor diye veri kalitesini/preprocessing'i önemsiz sanmak.
- CatBoost veya LightGBM defaultlarını domain kontrolü olmadan kullanmak.
- SHAP grafiği üretip business action çıkarmamak.
- Train score yüksek diye modeli başarılı saymak.

32. Mülakat soruları

Soru	Beklenen kısa cevap
Bagging ve boosting farkı?	Bagging paralel modelleri ortalayarak variance azaltır; boosting ardışık hata düzeltir.
Random Forest neden tek tree'den daha stabil?	Bootstrap + random feature sampling tree correlation'ını azaltır.
OOB nedir?	İlgili ağacın bootstrap sample'ına girmeyen gözlemler.
Boosting'de learning rate ne yapar?	Her yeni learner katkısını küçültür.
Early stopping neden kullanılır?	Validation iyileşmesi durunca overfit ve gereksiz round'ları azaltır.

Soru	Beklenen kısa cevap
XGBoost neden güçlü?	Regularized gradient boosting + second-order info + güçlü mühendislik.
LightGBM farkı?	Histogram ve leaf-wise growth ile hız/ölçek.
CatBoost ne zaman avantajlı?	Kategorik feature yoğun tabular veride.
Permutation importance ne ölçer?	Feature karıştırıldığında validation metric kaybını.
SHAP causal mı?	Hayır, model contribution açıklamasıdır.
Imbalance'da ne yaparsın?	PR-AUC, weighting, sampling, threshold ve cost-based evaluation.
Test set ne zaman kullanılır?	Final unbiased evaluation'da.

33. Uygulama alıştırmaları

1. Tek Decision Tree ile Random Forest'ı 20 farklı random seed altında karşılaştır; validation variance'ını ölç.
2. Random Forest `max_features` ve `min_samples_leaf` grid'ini dene.
3. Gradient Boosting için `learning_rate × n_estimators` karşılaştırması yap.
4. XGBoost early stopping kullanarak `best_iteration` bul.
5. LightGBM ve CatBoost'u aynı temporal split üzerinde kıyasla.
6. Imbalanced dataset'te ROC-AUC ve PR-AUC farkını raporla.
7. Impurity importance ve permutation importance sıralamalarını karşılaştır.
8. SHAP ile beş false-negative örneği incele.
9. Threshold'u business capacity ile optimize et.
10. Model latency ve model size'ı performance metricleriyle birlikte raporla.

34. Mini portföy projesi

E-Commerce Churn — Ensemble Benchmark & Explainability

Önerilen repo:

```
ensemble-churn-project/
|-- README.md
|-- data/
|-- notebooks/
|   |-- 01_problem_eda.ipynb
|   |-- 02_baselines.ipynb
|   |-- 03_random_forest.ipynb
|   |-- 04_boosting_models.ipynb
|   |-- 05_tuning.ipynb
|   |-- 06_explainability.ipynb
|   |-- 07_threshold_business.ipynb
|-- src/
|   |-- features.py
|   |-- train.py
|   |-- evaluate.py
|-- reports/
|   |-- executive_summary.pdf
|-- requirements.txt
```

CV'ye yazılabilecek güçlü ifade:

> Benchmarked Random Forest, XGBoost, LightGBM and CatBoost for churn prediction using temporal validation, early stopping, PR-AUC, threshold optimization and SHAP-based error analysis.

35. Final cheat sheet

Kavram	Hatırlanacak fikir
Bagging	Paralel modeller + averaging
Bootstrap	Replacement ile sampling
Random Forest	Bagging + random feature subset
OOB	Internal validation sinyali
Boosting	Ardışık hata düzeltme
`learning_rate`	Yeni tree katkısını küçültür
XGBoost	Regularized güçlü GBDT
LightGBM	Histogram + leaf-wise growth
CatBoost	Kategorik feature uzmanı
Early stopping	Validation durunca training'i kes
Subsample	Stochastic regularization
Permutation importance	Metric düşüşü üzerinden önem

Kavram	Hatırlanacak fikir
SHAP	Prediction contribution
PR-AUC	Rare positive için güçlü metric
Threshold	Business kararı
CV	Gerçek deployment koşullarını taklit etmeli

Bu modülden sonra

Bir sonraki başlık olan **Model Evaluation** bölümünde ensemble modelleri yalnızca "score" ile değil; doğru metric, calibration, threshold, confidence ve error analysis çerçevesiyle değerlendirmeye geçebilirsin.